Департамент
образования и науки
города Москвы

Государственное бюджетное
профессиональное образовательное
учреждение города Москвы

Колледж информационных
технологий «ИТ.МОСКВА»

IT.МОСКВА

8 499 612-03-25
spo-mgkeit@edu.mos.ru

115446, г. Москва,
ул. Академика Миллионщикова, д. 20

00216071 ОКПО
1027739491790 ОГРН
7724035635 ИНН
772401001 КПП

Практическая работа №4. Изучение работы в системе контроля версий

Специальность **09.02.07 Информационные системы и Программирование**

(код, наименование)

Вид практики Учебная практика УП.02

Сроки прохождения практики 29.09.26 - 12.10.26

Место прохождения практики колледж ИТ.МОСКВА

(название предприятия)

Выполнил(-а) студент(-ка) группы **3-ИП-7-24**

Муталибова Алина Руслановна

(Ф.И.О. студента)

Москва, 2026

1. Анализ предметной области

Предметная область — учебная операционная система с файловой системой на основе базы данных SQLite.

Разрабатываемая программа предназначена для управления файлами и пользователями. Она позволяет создавать файлы, сохранять и читать их содержимое, удалять файлы, а также создавать, просматривать и удалять учётные записи пользователей.

Для хранения информации используется SQLite. Данные о пользователях и файлах сохраняются в таблицах базы данных. Каждый файл связан с пользователем, которому он принадлежит.

Объектно-ориентированный подход позволяет разделить программу на классы, каждый из которых отвечает за определённую часть работы системы.

2. Определение функций предметной области

Основные функции программы:

– **Управление файлами:** создание, чтение, просмотр списка и удаление файлов.

– **Управление пользователями:** создание учётных записей, просмотр списка пользователей и удаление пользователей.

– **Хранение данных:** запись информации в SQLite и получение данных из базы.

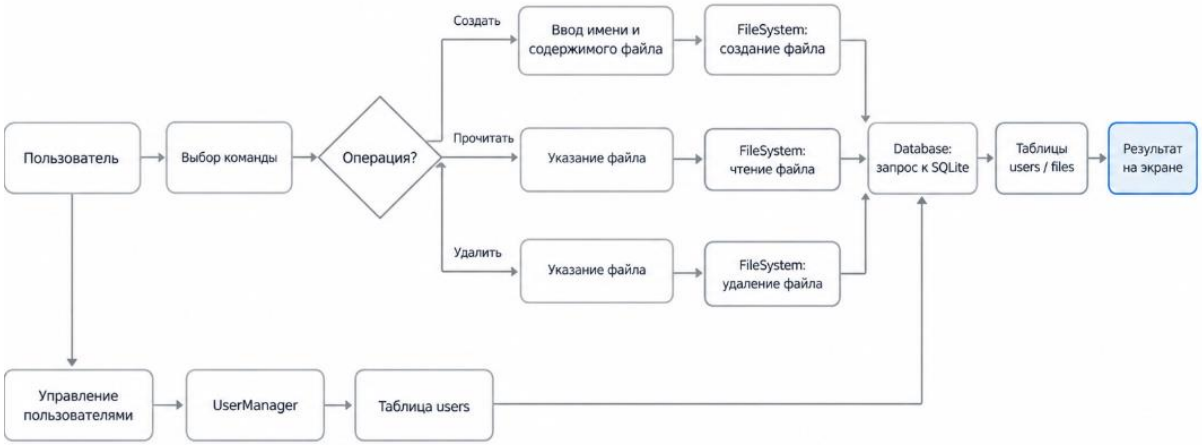
– **Проверка данных:** проверка существования пользователя или файла и обработка ошибок.

– **Взаимодействие с пользователем:** отображение меню, запрос необходимых данных и вывод результатов операций.

Программа должна сообщать пользователю об успешном выполнении операции или возникновении ошибки.

3. Схема документопотока

Пользователь выбирает необходимую команду: создание, чтение или удаление файла либо управление пользователями. При работе с файлами система принимает имя и содержимое файла или данные для его поиска. Запрос передаётся классу **FileSystem**, который выполняет соответствующую операцию и обращается к классу **Database** для взаимодействия с базой данных SQLite. Информация о файлах хранится в таблице **files**, а сведения о пользователях — в таблице **users**. Для управления пользователями используется класс **UserManager**.



После выполнения операции результат выводится на экран. Если файл или пользователь не найден либо введены некорректные данные, система выводит сообщение об ошибке.

4. Выделение сущностей, атрибутов и связей

В проекте выделены две основные сущности, хранящиеся в SQLite, и три программных класса, реализующих бизнес-логику.

Сущность	Атрибуты	Назначение
Пользователь (User)	user_id — идентификатор; username — имя пользователя	Хранение учётных записей.
Файл (File)	file_id — идентификатор; name — имя; content — содержимое; owner_id — владелец; created_at — дата создания	Хранение файлов и связи с их владельцами.
Database	connection — соединение с SQLite	Создание таблиц и выполнение запросов к базе.
UserManager	database — объект Database	Добавление пользователей и получение информации о них.
FileSystem	database — объект Database	Создание, чтение, удаление и перечисление файлов.

Связи между сущностями:

– **Пользователь — Файл**: связь «один ко многим» (1:N). Один пользователь может владеть несколькими файлами.

– **Файл — Пользователь**: каждый файл связан с одним владельцем через внешний ключ owner_id.

– **FileSystem** и **UserManager** используют Database для работы с одной базой SQLite.

Таким образом, таблица users содержит пользователей, а таблица files файлы. Поле owner_id связывает запись файла с конкретным пользователем.

5. Концептуальная модель

Программный продукт строится на разделении ответственности между классами. Класс **Database** отвечает за подключение к базе данных SQLite, создание таблиц и выполнение запросов. Класс **UserManager** предназначен для добавления пользователей и получения информации о них, а класс **FileSystem** выполняет операции с файлами: создание, чтение, удаление и вывод списка файлов.

UserManager	FileSystem	Database	Пользовательский интерфейс
add_user() get_user()	create_file() read_file() delete_file() list_files()	connect() create_tables() execute()	Принимает команды, передаёт запросы и показывает результат

Пользовательский интерфейс принимает команды пользователя, передаёт запросы соответствующим классам и отображает результаты выполнения операций. Классы **UserManager** и **FileSystem** используют общий объект **Database** для доступа к базе SQLite, в которой хранятся сведения о пользователях и файлах. Такое разделение упрощает структуру программы и позволяет изменять логику работы с файлами и пользователями независимо от механизма хранения данных.

Основные принципы ООП

Принцип	Как проявляется в проекте

Класс и объект	Database, UserManager и FileSystem описывают отдельные части системы; в программе создаются их объекты.
Инкапсуляция	Операции с базой и файлами выполняются через методы классов, а не напрямую из интерфейса.
Абстракция	Пользователь вызывает понятные операции create_file(), read_file() и delete_file(), не работая с SQL-запросами напрямую.
Разделение ответственности	Класс Database отвечает за хранение, UserManager — за пользователей, FileSystem — за файлы.

6. Описание входных и выходных данных

Входные данные

Для работы с учебной операционной системой пользователь вводит команды через интерфейс программы. В зависимости от выбранного действия система получает следующие данные:

- **При создании файла** — имя файла, его содержимое и идентификатор владельца.
- **При чтении файла** — идентификатор или имя файла, который необходимо открыть.
- **При удалении файла** — данные, позволяющие определить нужный файл.

– **При работе с пользователями** — имя нового пользователя или идентификатор пользователя для поиска.

– **При просмотре списка файлов** — идентификатор пользователя, файлы которого необходимо отобразить.

Перед выполнением операций программа проверяет корректность введённых данных и наличие соответствующих записей в базе SQLite.

Выходные данные

В результате работы программы пользователь получает информацию о выполненной операции. Система может выводить:

– подтверждение успешного создания или удаления файла;

– содержимое выбранного файла;

– список файлов, принадлежащих пользователю;

– сведения о найденном пользователе;

– сообщение об ошибке, если файл или пользователь отсутствует либо данные введены некорректно.

Все результаты формируются на основе данных, хранящихся в базе SQLite, и отображаются пользователю через интерфейс программы.

Пример: пользователь добавляет учётную запись «student1», создаёт файл «notes.txt» с текстом «Мой первый файл», затем читает его. Система сохраняет запись в SQLite и при чтении выводит содержимое. При удалении файла система сообщает об успешном выполнении операции.

Вывод

В ходе практической работы выполнено объектно-ориентированное проектирование учебной операционной системы с файловой системой на SQLite. Проведён анализ предметной области, определены функции

программы, описан документопоток, выделены сущности и связи, составлена концептуальная модель, определены входные и выходные данные. Разделение системы на классы **Database**, **UserManager** и **FileSystem** упрощает сопровождение программы и позволяет в дальнейшем добавлять новые операции с файлами и пользователями.

Разработанная модель позволяет организовать хранение файлов и управление пользователями с помощью объектно-ориентированного подхода и базы данных SQLite.